

LAPORAN PRAKTIKUM INTERNET OF THINGS

Dosen pengampu : Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

Andi Saptono, S.Kom



Disusun oleh : Dina septiyani 25782038

POLITEKNIK NEGERI LAMPUNG

JURUSAN TEKNOLOGI INFORMASI

PROGRAM STUDI TEKNOLOGI REKAYASA INTERNET

2026

4.6 Latihan

Latihan 1 : Penggunaan teknologi **WebSockets** terbukti jauh lebih efisien dalam konsumsi *bandwidth* dan menjaga kestabilan antarmuka dibandingkan metode *Meta Refresh HTTP*. Berbeda dengan *Meta Refresh* yang memuat ulang (*reload*) seluruh halaman web secara berkala beserta seluruh aset pendukungnya—sehingga boros data dan memicu kedipan layar (*flickering*)—WebSockets mempertahankan satu koneksi persisten (*full-duplex*) di latar belakang dan hanya mengirimkan paket data perubahan (*payload*) secara asinkron. Hal ini memungkinkan pembaruan data secara *real-time* melalui manipulasi DOM tanpa memuat ulang halaman, sehingga penggunaan kuota internet lebih hemat dan tampilan visual layar tetap mulus tanpa kedipan.

Latihan 2 (Analisis Waktu/Timer): Penggunaan fungsi `delay(3000)` di dalam `void loop()` akan menyebabkan **blokada total** (*blocking*) pada mikrokontroler. Akibatnya, NodeMCU gagal memproses sinyal *heartbeat* WebSocket, yang berujung pada **putusnya koneksi secara otomatis** (*timeout*), membuat sistem tidak responsif, dan menghentikan pembaruan data *real-time*.

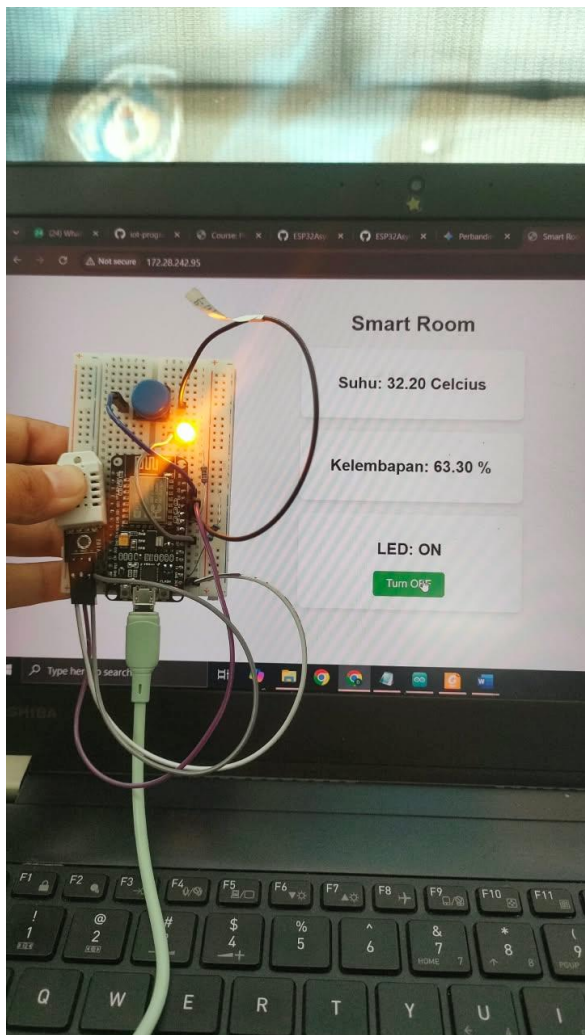
Latihan 3 (Modifikasi Front-End): Karena nama ID elemen HTML bergantung pada rancangan *front-end* dashboard IoT Anda (seperti proyek Smart Warehouse NodeMCU ESP8266), berikut adalah contoh umum nama ID `<span id="...">` yang biasanya menjadi target injeksi teks atau status perangkat dalam blok skrip JavaScript tersebut:

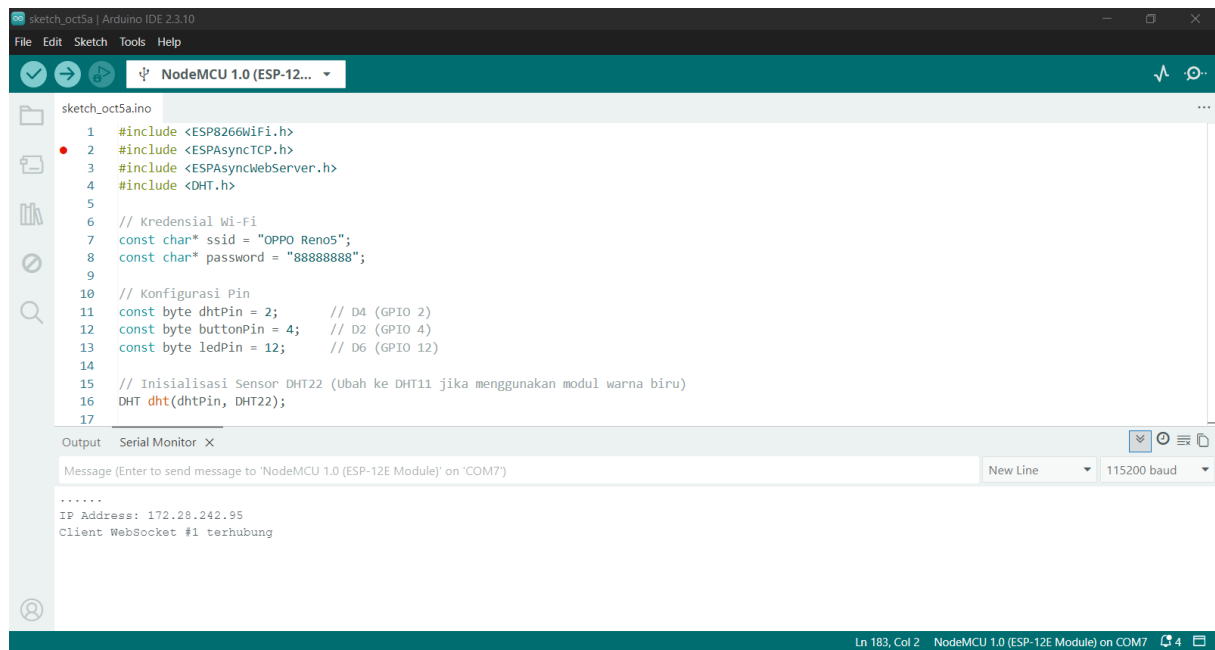
- **status** atau **connection-status**: Untuk menampilkan status koneksi WebSocket (*Connected / Disconnected*).
- **relay-status** atau **lamp-status**: Untuk menampilkan status kondisi aktuator atau perangkat keras (*ON / OFF*).
- **temp-status** / **humidity-status**: Untuk menampilkan data pembacaan sensor (seperti DHT22).
- **ldr-status**: Untuk menampilkan status sensor cahaya.

Latihan 4 (Pengembangan Telemetry Sensor - Kelembapan):

Pada pengembangan Latihan 4, sistem antarmuka web *Smart Room* berhasil ditambahkan fitur pembacaan data kelembapan (*humidity*) secara *real-time*. Sisi *back-end* NodeMCU membaca nilai kelembapan dari sensor DHT22 menggunakan fungsi `dht.readHumidity()` setiap interval 3 detik, lalu mengemasnya bersama data suhu dan status LED ke dalam format JSON

Pengujian Latihan 4 menunjukkan keberhasilan integrasi telemetry ganda dan kontrol aktuator pada antarmuka web *Smart Room*. Sensor DHT22 berhasil membaca parameter lingkungan secara *real-time* dengan suhu terukur sebesar **32.20 Celcius** dan kelembapan **63.30 %**. Data telemetry tersebut dikirimkan oleh NodeMCU melalui protokol WebSocket sehingga tampil secara sinkron pada layar monitor. Pada saat bersamaan, fitur kontrol aktuator berjalan normal yang ditandai dengan status `LED: ON` di antarmuka web dan menyalanya indikator LED fisik pada *breadboard*.





Pengujian selanjutnya pada Latihan 4 menunjukkan kondisi sistem ketika beban dinonaktifkan (LED: OFF). Saat tombol Turn OFF pada antarmuka web diklik atau push button fisik ditekan, sinyal pemutusan perintah dikirimkan secara *real-time* via WebSocket untuk mematikan pin kontrol D6 pada NodeMCU, sehingga indikator LED fisik pada *breadboard* padam dan status antarmuka web secara otomatis berubah menjadi LED: OFF dengan tombol bertuliskan Turn ON. Di saat yang sama, pemantauan telemetry lingkungan tetap berjalan aktif dengan pembacaan sensor DHT22 yang diperbarui secara berkala, yaitu suhu sebesar **31.60 Celcius** dan kelembapan **61.60 %**.

